

Part 1

1. `p` is an integer pointer, refers to the address of an integer object. `pp` is the pointer for an integer pointer, refers to the address of an address. `*pp` dereferences `pp`, and would be the integer pointer equal to `p`. `**pp` is the integer found after dereferencing `p`, the integer value at location `p`.
2. `p` is an integer pointer, stores the memory address of an integer. `&p` is the address of `p` itself, aka the address at which the integer's address is stored. `*p` dereferences `p`, aka the integer value stored at address `p`. `a` is an integer array with 4 elements, and `&a` is the memory address for the start of that array.
3. `p+1` is the address of the object immediately after `p`. 1 in this case refers to the size of the type of object that `p` is pointing to. for example if `p` is a `char`, `p+1` would be the next immediate `char` after `p`, aka the numeric address `A+1`. if `p` is a `integer`, 4 bytes, then `p+1` would be `A+4`.
4. 10 30 30 (not 10 20 6, forgot to consider the `update()` function)
5. `a:` stack
`p:` stack
`pp:` stack
`local:` stack
`heap:` stack
`*heap:` heap
6. easier to maintain as it's one less thing to go wrong. `sizeof(heap)` would measure the number of bytes that the heap value itself is, aka the address. `sizeof(*heap)` would measure the size of the object that is stored at the address of heap.

Part 2

After a successful build, the Makefile would go through and see that nothing has changed.

therefore no need to recompile. Changing `stats.c` and recompiling would only touch `stats.o` and `stats`, keeping `main.o` the same as nothing needs to change there.

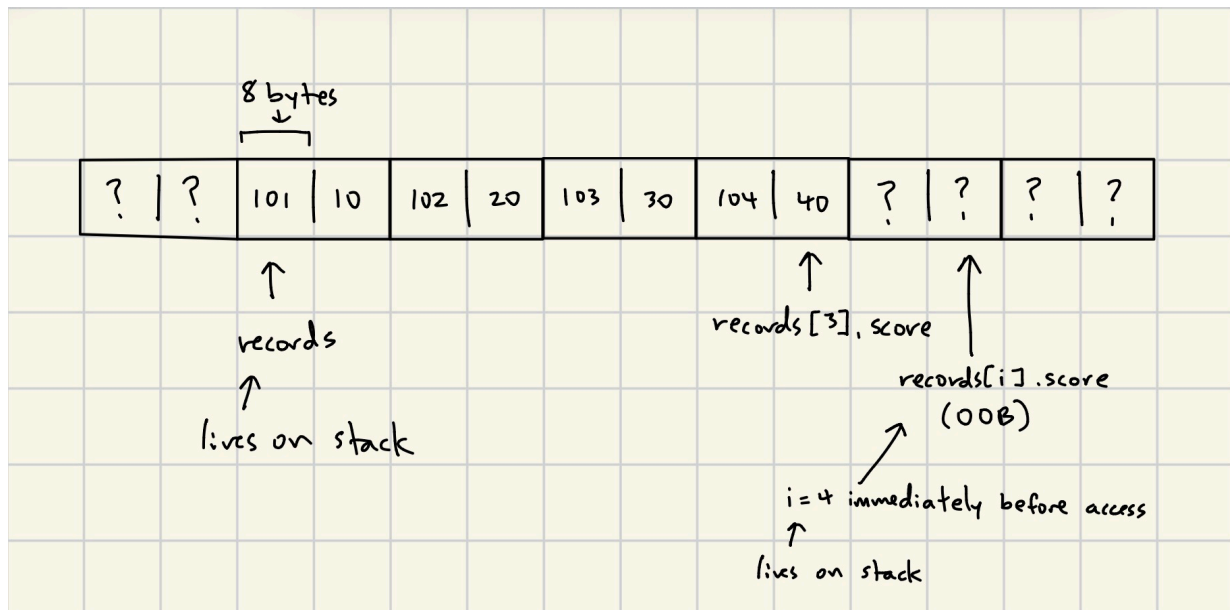
Changing `stats.h` means both `stats.o` and `main.o` would need to be recompiled, and obviously so would `stats`. Therefore everything would need to be recompiled.

Part 3

I start off with looking over the program, identifying that total is supposed to be 100, and then running the program. Looked fine, no errors, then took a look at total_scores() and used 'n' to go through line by line. Because of the "<=" in the for loop condition, the "+=" ran 5 times, which is incorrect. This did not end up as a problem on my end because coincidentally records[4], an invalid/OOB memory address just so happened to be 0. Records[4] is obviously invalid as the array is only of size=4.

(after 4 loops...)

```
12     for (int i = 0; i <= n; i++) {  
(gdb) n  
13         total += records[i].score;  
(gdb) print i  
$1 = 4  
(gdb) x &records[i]  
0x6022c0: 0x00000000  
(gdb) x &records[4]  
0x6022c0: 0x00000000  
(gdb) print records[i]  
$2 = {id = 0, score = 0}
```



Part 4

1. The printed "&local" addresses correspond to the stack objects whereas the printed "heap" addresses correspond to the heap objects. I know because in the program the "heap" objects are initialized with malloc() whereas the "local" is just a local variable.
2. On the OS for the cse machines the stack grows downwards whereas the heap grows upwards.
3. Correct, the stack and heap objects are in the expected regions:

```
cat /proc/974467/maps
00d2a000-00d4b000 rw-p 00000000 00:00 0 [heap]
7ffc37f41000-7ffc37f63000 rw-p 00000000 00:00 0 [stack]
```

```
depth= 0 &local=0x7ffc37f60224 heap=0xd2a2a0
```

4. Stack = 0x22000, 139,264 bytes. Heap = 0x21000, 135,168 bytes. These are the virtual memory available to the process, not necessarily the actual physical RAM being used up.
5. execve() - runs the specified executable
write() - called by printf()
getpid() - grabs the PID of the current process
brk() - called when malloc() needs more heap space
read() - reads data from a file and load it into memory, used to wait for the "enter" press from cmd line

Part 5

Exec	Output
./memory-bugs 1	test1: 30
./memory-bugs 2	test2: 30
./memory-bugs 3	test3: 2265
./memory-bugs 4	test4: operating systems

Testcase	Source	Error	Fix
1	for (size_t i = 0; i <= n; ++i)	invalid write	"i <= n" should be "i < n"
2	for (size_t i = 0; i < count; ++i)	invalid write	For loop should be "for (size_t i = 0; i <= count; i++)"
2	printf("test2: %d\n", a[0] + a[1] + a[n - 1]);	a[0] is uninitialized	Resolved after fixing above for loop
3	printf("test3: %d\n", *p);	tried to read *p after freeing	Run before release_int() is called
4	}	heap allocated "message" was never freed	Add free(message) before the end of the function

Part 6

1. Everything is equal, global, stack, and heap addresses.
PARENT_AFTER_FORK pid=978629 child=978630 global=0x60207c
stack=0x7ffc403968a0 heap=0x1ee72a0 values=100,200,300
CHILD_AFTER_FORK pid=978630 ppid=978629 global=0x60207c
stack=0x7ffc403968a0 heap=0x1ee72a0 values=100,200,300
No, the same virtual addresses do not necessarily refer to the same physical memory location. They still have different address spaces, and if either one modifies the memory data, they will no longer point to the same physical memory locations.
2. No, after the child write the values in parent don't change because the OS makes a separate copy for each. They start out as the same through copy-on-write, but the changed pages get copied to a different location on write.
3. CHILD_AFTER_FORK pid=979110 ppid=979109 global=0x60207c
stack=0x7ffd12263740 heap=0x1e932a0 values=100,200,300
AFTER_EXEC pid=979110 global=0x60105c stack=0x7ffc0a8a8ac4 heap=0xfe52a0
values=900,901,902

```
e5-cse-135-10 3$ cat /proc/979110/maps
01e93000-01eb4000 rw-p 00000000 00:00 0           [heap]
7ffd12245000-7ffd12267000 rw-p 00000000 00:00 0           [stack]
```

```
e5-cse-135-10 4$ cat /proc/979110/maps
00fe5000-01006000 rw-p 00000000 00:00 0           [heap]
7ffc0a889000-7ffc0a8ab000 rw-p 00000000 00:00 0           [stack]
```

After exec the child only retains the same PID. The old address space gets wiped and replaced with a new one.

4. **fork/clone** - clone(child_stack=NULL,
flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, strace: Process
979288 attached, child_tidptr=0x7fef7c6b28d0) = 979288

Child exec - [pid 979288] execve("./child", ["child"], 0x7fff2448acc8 /* 52 vars */) = 0

Parent wait - [pid 979287] wait4(979288, <unfinished ...>

Parent wait resumed - [pid 979288] <... read resumed>"\\n", 1024) = 1
<... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 979288
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=979288,
si_uid=5669221, si_status=0, si_etime=0, si_stime=0} ---